

CO2-AT2 : Visualization Development Exercise

VISHAAL M S

Register Number: 192424420

Date: 03/08/2026

Question 1

Visualizing a Single Distribution

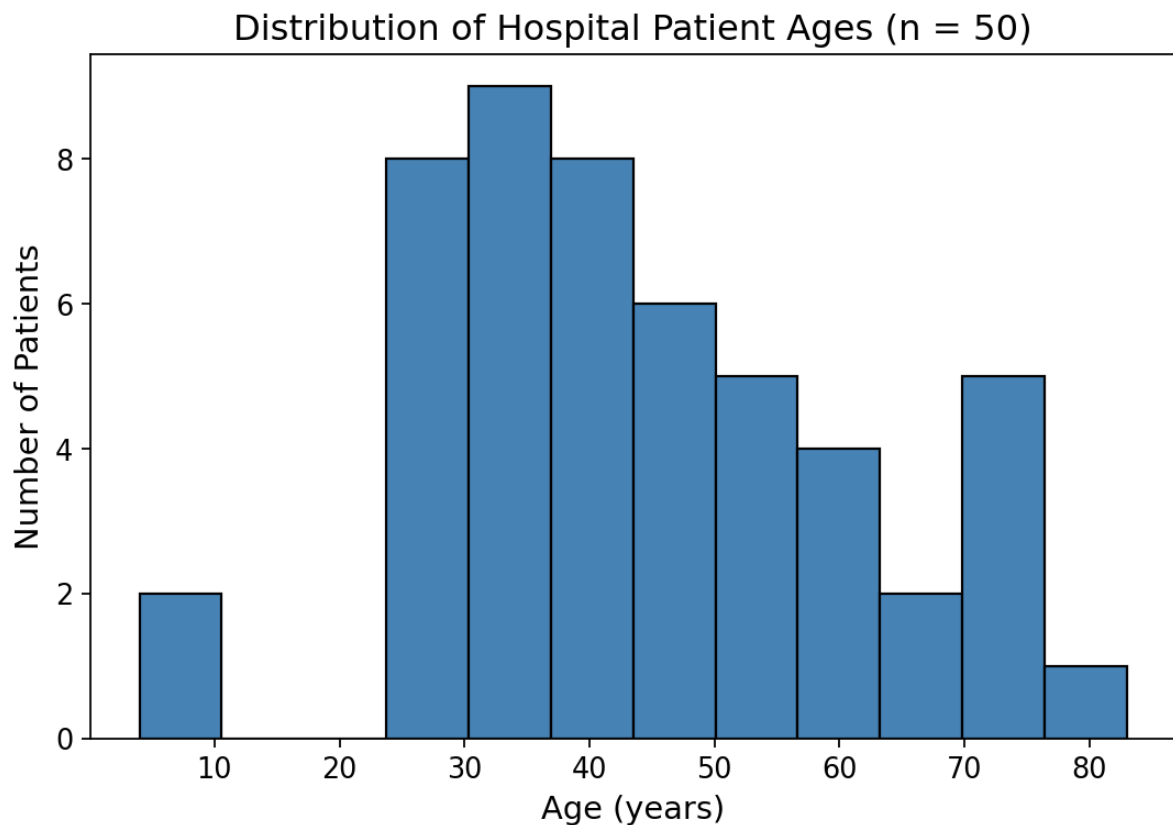
Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated dataset: ages of 50 hospital patients
np.random.seed(1)
patient_ages = np.round(np.random.normal(loc=45, scale=18, size=50))
patient_ages = np.clip(patient_ages, 1, 95)

# Histogram is the standard chart for summarizing a single distribution's
# overall shape (center, spread, skew) at a glance.
plt.figure(figsize=(7, 5))
plt.hist(patient_ages, bins=12, color='steelblue', edgecolor='black')
plt.title("Distribution of Hospital Patient Ages (n = 50)")
plt.xlabel("Age (years)"); plt.ylabel("Number of Patients")
plt.tight_layout()
plt.show()
```

Output



Question 2

Comparing Multiple Distributions on One Chart

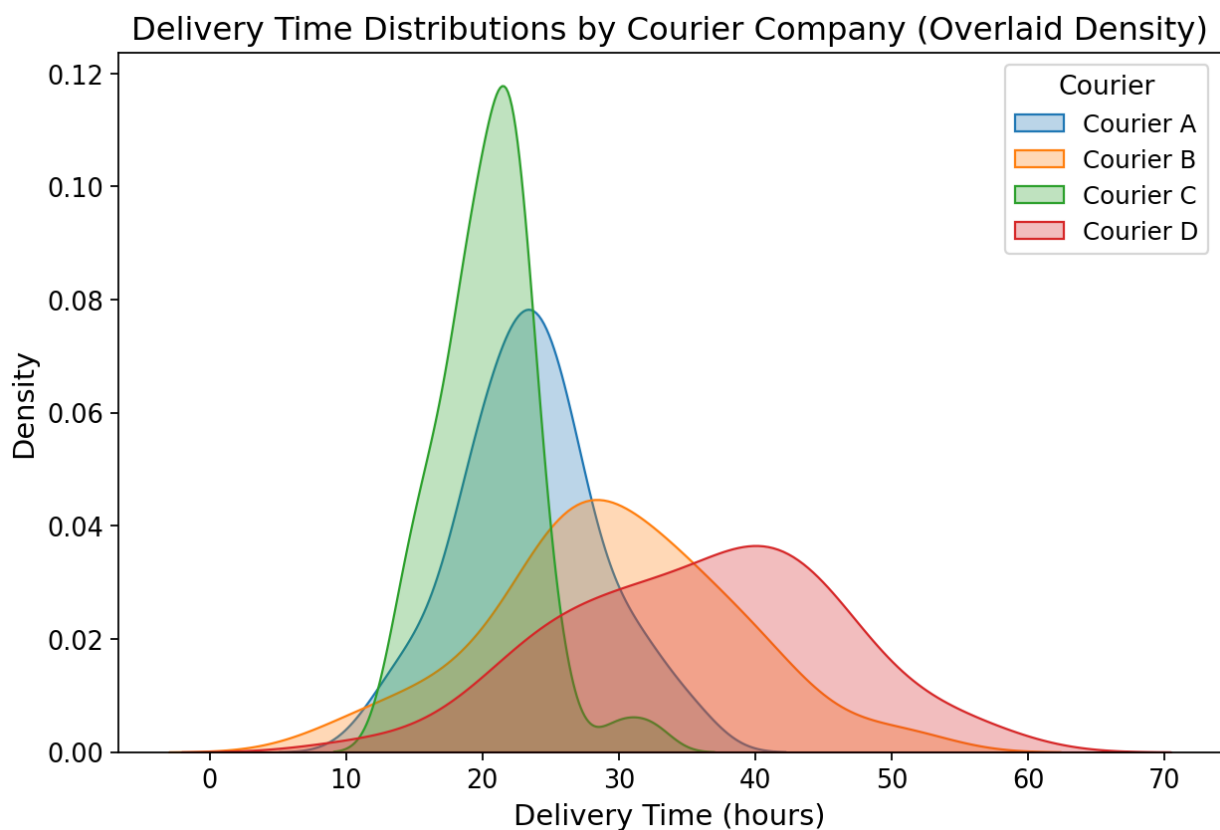
Code (Python)

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Simulated dataset: delivery times (hours) for 4 courier companies
np.random.seed(2)
n = 60
data = {
    'Courier A': np.random.normal(24, 5, n),
    'Courier B': np.random.normal(30, 8, n),
    'Courier C': np.random.normal(20, 3, n),
    'Courier D': np.random.normal(36, 10, n),
}
df = pd.concat([pd.DataFrame({'DeliveryTime': np.clip(v, 1, None), 'Courier': k})
                for k, v in data.items()], ignore_index=True)

# Overlaid density plot lets all four companies' distributions be compared
# directly on one shared axis.
plt.figure(figsize=(8, 5.5))
for courier in data.keys():
    sns.kdeplot(df[df['Courier'] == courier]['DeliveryTime'], label=courier, fill=True, alpha=0.3)
plt.title("Delivery Time Distributions by Courier Company (Overlaid Density)")
plt.xlabel("Delivery Time (hours)"); plt.ylabel("Density")
plt.legend(title="Courier")
plt.tight_layout()
plt.show()
```

Output



Question 3

Boxplots with Outliers Marked Explicitly

Code (Python)

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Simulated dataset: delivery times (hours) for 4 delivery partners
np.random.seed(3)
n = 50
data = {
    'Partner A': np.random.normal(22, 4, n),
    'Partner B': np.random.normal(28, 6, n),
    'Partner C': np.random.normal(25, 3, n),
    'Partner D': np.random.normal(33, 9, n),
}
df = pd.concat([pd.DataFrame({'DeliveryTime': np.clip(v, 1, None), 'Partner': k})
                for k, v in data.items()], ignore_index=True)

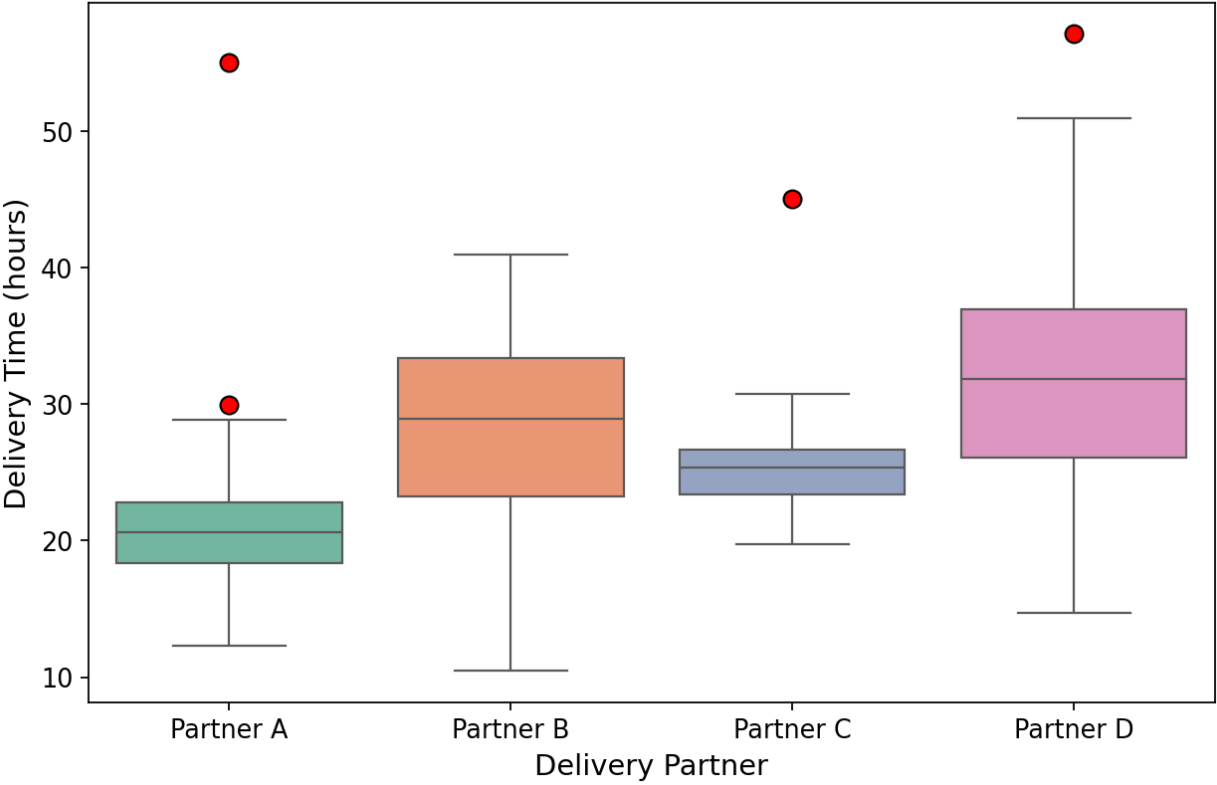
# inject a couple of clear outliers to demonstrate outlier marking
df.loc[df['Partner'] == 'Partner A'].index[0], 'DeliveryTime' = 55
df.loc[df['Partner'] == 'Partner C'].index[0], 'DeliveryTime' = 45

plt.figure(figsize=(8, 5.5))
ax = sns.boxplot(data=df, x='Partner', y='DeliveryTime', hue='Partner',
                 palette='Set2', legend=False,
                 flierprops=dict(marker='o', markerfacecolor='red', markersize=8,
                                   markeredgecolor='black'))
plt.title("Delivery Time Comparison by Partner (Outliers Marked in Red)")
plt.xlabel("Delivery Partner"); plt.ylabel("Delivery Time (hours)")
plt.tight_layout()
plt.show()

# Explicitly report outliers per partner using the 1.5xIQR rule
for partner in data.keys():
    sub = df[df['Partner'] == partner]['DeliveryTime']
    q1, q3 = sub.quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    outliers = sub[(sub < lower) | (sub > upper)]
    print(f"{partner} - outliers: {outliers.round(1).tolist() if len(outliers) else 'none'}")
```

Output

Delivery Time Comparison by Partner (Outliers Marked in Red)



Question 4

Amounts – Sorted Bar Chart with Custom Palette

Code (Python)

```
import matplotlib.pyplot as plt
import pandas as pd

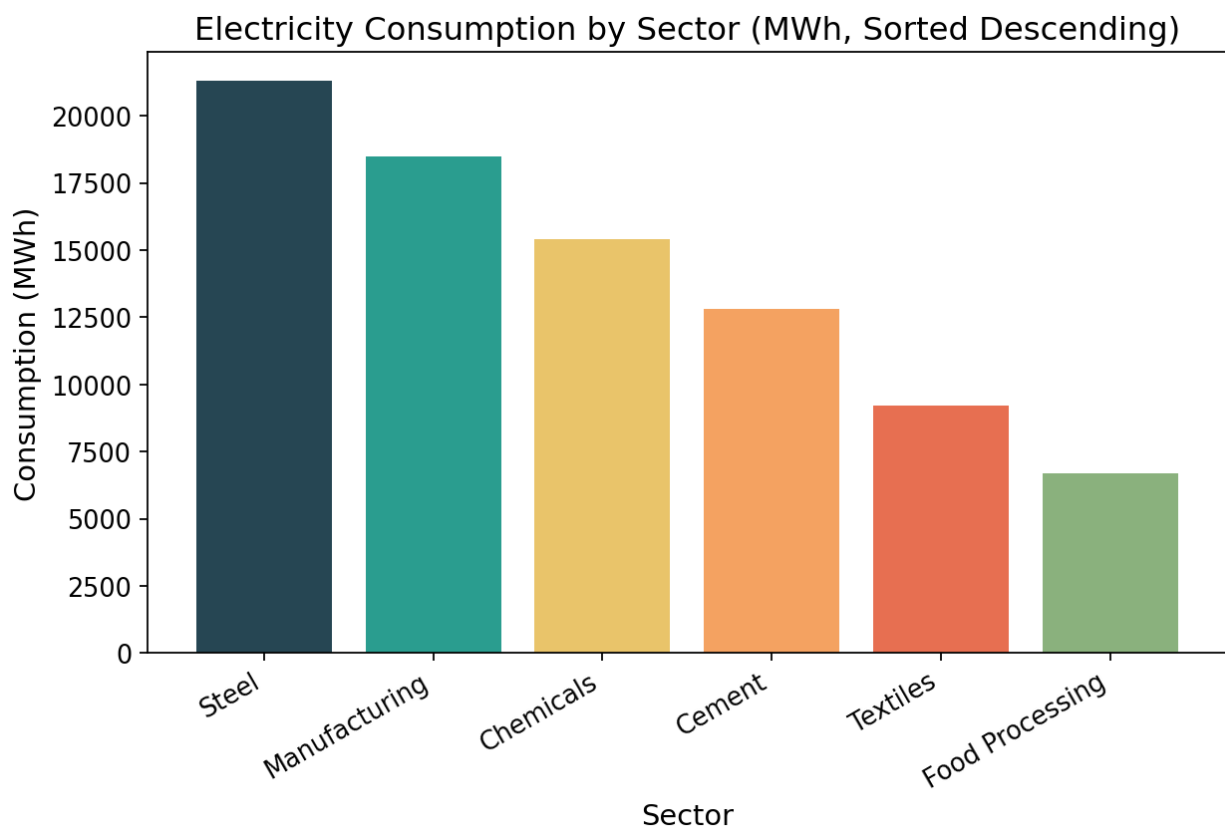
# Small hardcoded dataset: electricity consumption (MWh) for 6 sectors,
# read from a dictionary into a DataFrame
consumption_dict = {
    'Manufacturing': 18500,
    'Textiles': 9200,
    'Chemicals': 15400,
    'Steel': 21300,
    'Cement': 12800,
    'Food Processing': 6700,
}
df = pd.DataFrame(list(consumption_dict.items()), columns=['Sector', 'Consumption_MWh'])

# Sort descending before plotting
df = df.sort_values('Consumption_MWh', ascending=False)

# Custom color palette (one distinct color per sector)
custom_colors = ['#264653', '#2A9D8F', '#E9C46A', '#F4A261', '#E76F51', '#8AB17D']

plt.figure(figsize=(8, 5.5))
plt.bar(df['Sector'], df['Consumption_MWh'], color=custom_colors)
plt.title("Electricity Consumption by Sector (MWh, Sorted Descending)")
plt.xlabel("Sector"); plt.ylabel("Consumption (MWh)")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```

Output



Question 5

Distribution of a Numeric Measurement Across a Group

Code (Python)

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Simulated dataset: ages of 45 employees, generated from a normal distribution
np.random.seed(5)
employee_ages = np.round(np.random.normal(loc=34, scale=8, size=45))
employee_ages = np.clip(employee_ages, 21, 60) # keep ages within a realistic working range

plt.figure(figsize=(7, 5))
sns.histplot(employee_ages, bins=10, stat='density', color='mediumpurple',
             edgecolor='black', alpha=0.6)
sns.kdeplot(employee_ages, color='black', linewidth=1.5)
plt.title("Distribution of Employee Ages (n = 45)")
plt.xlabel("Age (years)"); plt.ylabel("Density")
plt.xlim(18, 65) # sensible working-age range, avoids a misleadingly wide axis
plt.tight_layout()
plt.show()
```

Output

